

LAB REPORT: PROCESSOR DESIGN AND IMPLEMENTATION

NAME: EMAAN ASAD 09904

LAMIA MUNIR SHAIKH 09977

1. Introduction

This lab involved the comprehensive design, verification, and implementation of a 32-bit RISC-V RV32I single-cycle processor. The project focused on developing fundamental hardware modules and integrating them into a functional system capable of executing assembly programs. Key modules designed and verified include the Register File, Arithmetic Logic Unit (ALU), Instruction Memory, Immediate Generator, Control Unit, and Data Memory. Following the integration of these components, the processor was synthesized and implemented on an FPGA to perform real-world tasks through assembly instruction execution.

2. Task A: Running Assembly Program

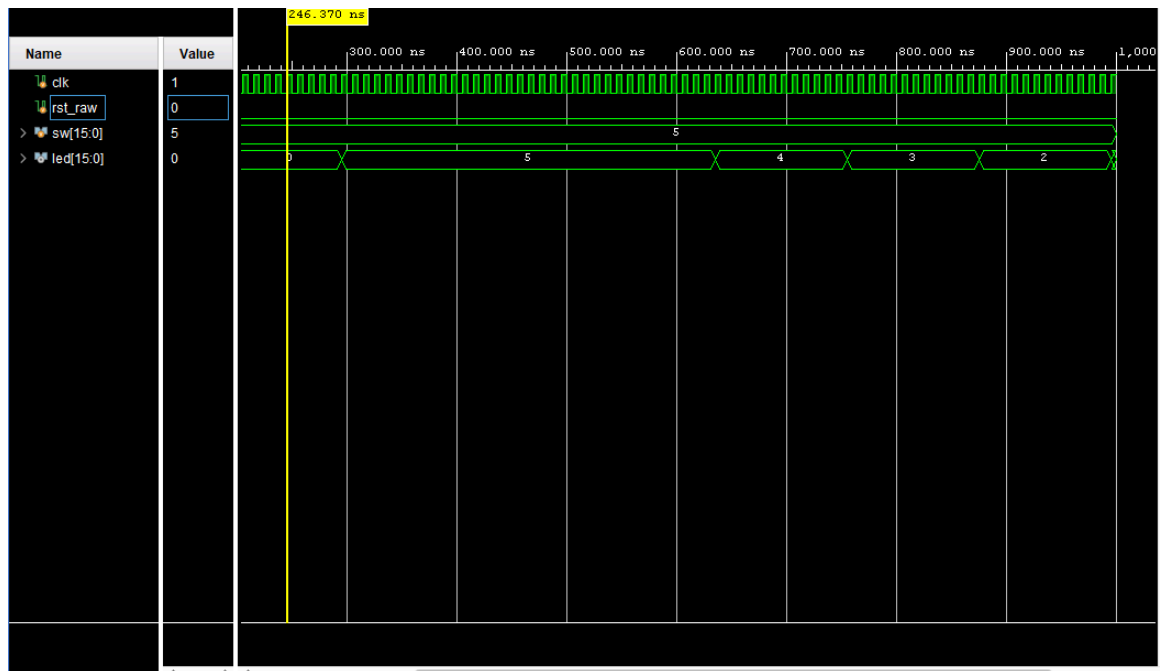
- **Module Development:** All primary RTL modules, including the 32-bit ripple carry adder-based ALU, a 32x32-bit Register File, and the Instruction Memory, were developed and independently verified.
- **System Integration:** Modules were integrated into a TopLevelProcessor design, which was subsequently wrapped in a ProcessorFPGA module for hardware compatibility.
- **Memory Loading:** Assembly programs, such as the countdown and Fibonacci sequences, were converted into hexadecimal format and loaded into the instruction memory using .mem files.
- **Simulation and Verification:** A testbench (tb_Countdown) was utilized to simulate the processor's behavior, applying switch inputs and observing LED outputs.
- **Hardware Implementation:** Physical constraints were defined in the FPGA_constraints.xdc file, mapping processor signals like clk, rst_raw, sw, and led to the appropriate FPGA pins.
- **Execution:** The processor successfully executed the sequential logic, including conditional branches and register updates, as confirmed by waveform analysis.

The datapath as shown in the schematic includes the following critical modules and logic flows:

- This schematic demonstrates the hardware-level integration required to successfully execute the assembly programs verified in this lab, such as the Fibonacci and countdown sequences.

This schematic demonstrates the hardware-level integration required to successfully execute the assembly programs verified in this lab, such as the Fibonacci and countdown sequences.

Waveform:



3. Task B: New Instruction Implementation

Instruction Descriptions

- **LUI:** The LUI (Load Upper Immediate) instruction was implemented to build 32-bit constants. It loads a 20-bit immediate into bits [31:12] of the destination register, zeroing the lower bits.
- **JAL:** The JAL (Jump and Link) instruction handles unconditional jumps. It adds a 20-bit signextended offset to the PC for the jump and stores PC+4 in the link register x1.
- **AUIPC:** AUIPC (Add Upper Immediate to PC) is a U-type instruction used to build PC-relative addresses by adding a 20-bit shifted immediate to the current PC.

Modifications to the processor:

To support the addition of the **LUI**, **AUIPC**, and **JAL** instructions to the RV32I datapath, several structural and logic-level modifications were made to the core architecture. These changes ensure that the processor can handle upper immediates, PC-relative addressing, and unconditional jumps with link-back capability.

1. Datapath Modifications and Extensions

The datapath was enhanced to provide new routing for the Program Counter (PC) and the Immediate Generator (ImmGen) results.

- **Immediate Generator Expansion (U-type & J-type):**

- The ImmGen module was updated to extract the 20-bit immediate for **LUI** and **AUIPC** (bits) and the 20-bit jumped immediate for **JAL**.
- For **LUI** and **AUIPC**, this immediate is shifted left by 12 bits to align with the upper 20 bits of a 32-bit word.
- **PC Target Logic (JAL/AUIPC):**
 - A **Branch Adder** (as seen in branchAdder.v) was integrated to calculate PC + imm.
 - Unlike conditional branches, for **JAL**, this result is fed directly into the PC multiplexer, bypassing the "Zero" flag condition.
 - For **AUIPC**, this adder result is routed to the write-back multiplexer to be stored in the destination register (rd).
- **Write-back Multiplexer (RegWrite Mux):**
 - The multiplexer at the end of the datapath was expanded from a 2-to-1 to a **4-to-1 multiplexer**. It now selects between:
 1. **ALU Result:** For R-type and I-type (like addi).
 2. **Data Memory:** For lw instructions.
 3. **PC + 4:** For **JAL** to save the return address in x1.
 4. **Upper Immediate / PC-Relative Result:** For **LUI** and **AUIPC**.

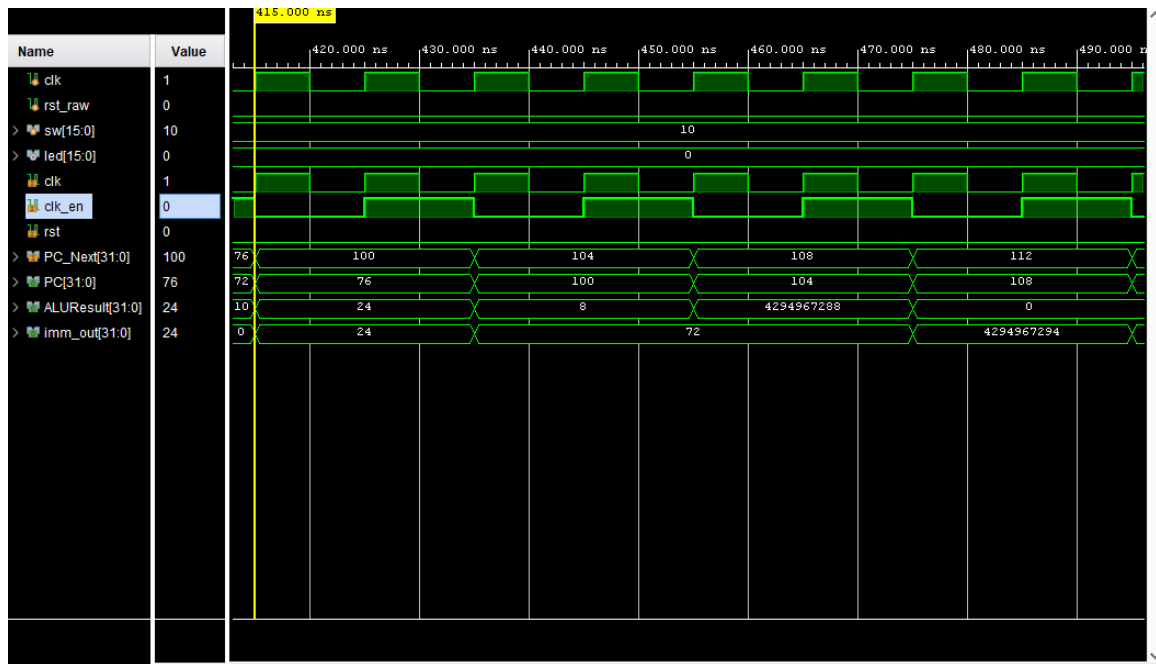
2. Control Signal and Control Path Modifications

The Control Unit was redesigned to generate specific steering signals for the new instruction formats. The primary change was the introduction of a more complex MemtoReg (or ResultSrc) signal and a Jump signal.

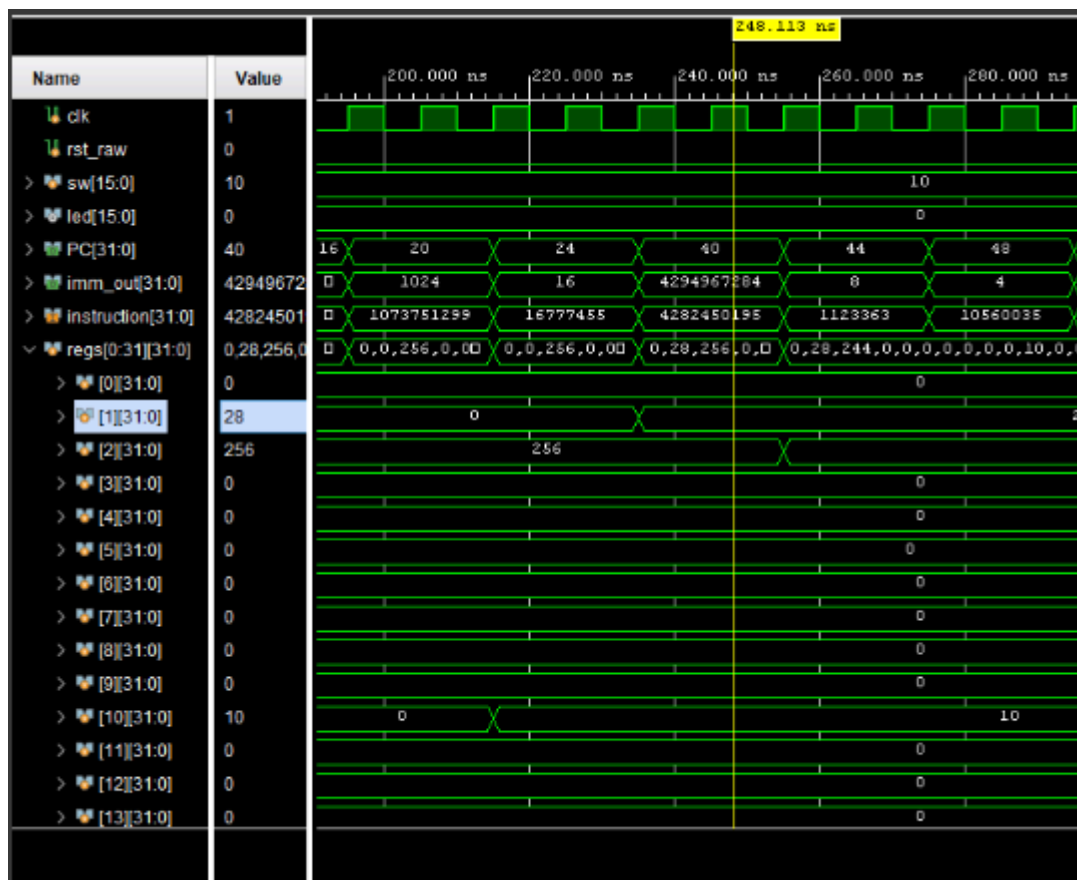
- **New Control Signals:**
 - **Jump:** Asserted for **JAL**. This signal forces the PC source multiplexer to select the Branch Adder result, regardless of ALU flags.
 - **ALUOp [00]:** As shown in your ALUControl.v, the ALUOp is set to 2'b00 for these instructions, forcing the ALU to perform an **ADD** operation for address or immediate calculations.

Verification Details

LUI Verification: LUI was verified by monitoring the destination register in the simulation of Task A as well as Task B. The waveform confirmed the immediate was shifted left by 12 bits and stored correctly while lower bits remained zero. The **ALUResult** and **imm_out** signals demonstrate successful offset calculations and immediate extraction that are reflected by both signals.

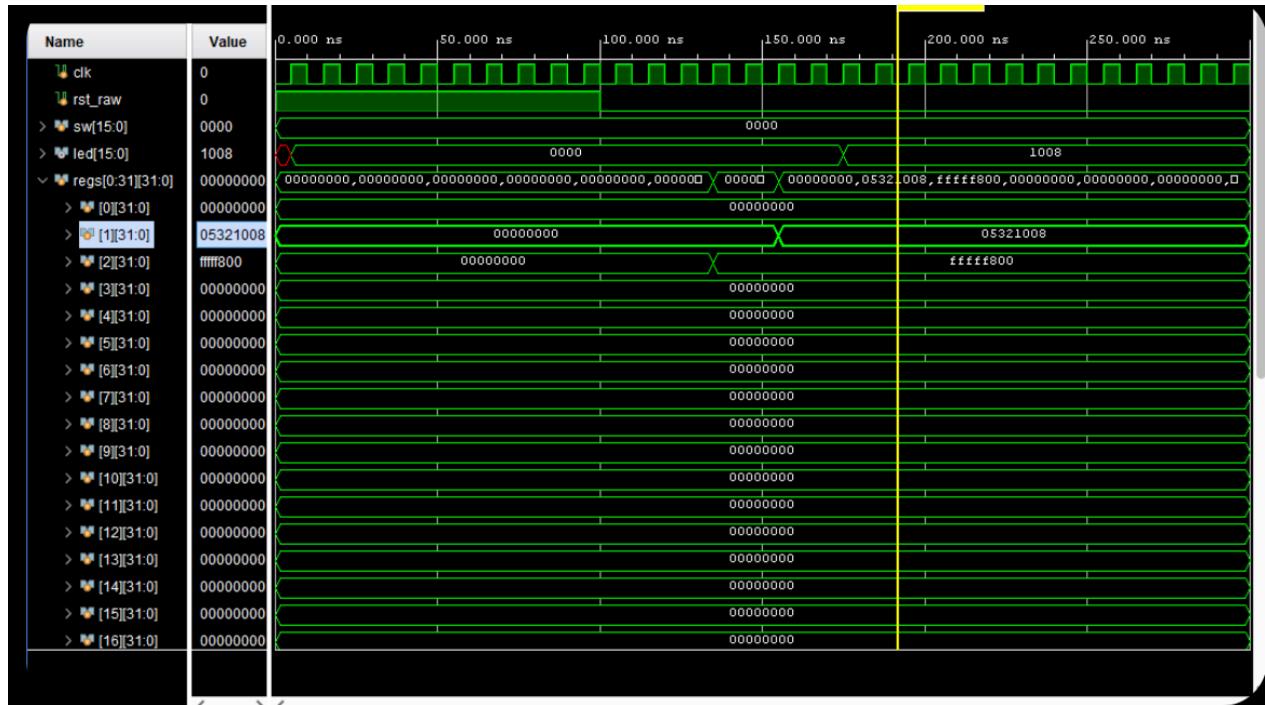


JAL Verification: JAL was verified by observing the PC jump to the target address calculated from the offset while simultaneously saving the return address in x1 in the waveform of task B. Attached below is the waveform of task B that shows a jump in PC from 24 to 40 as calculated by the immGen as 16. The address of the next instruction, 28, is saved in x1.



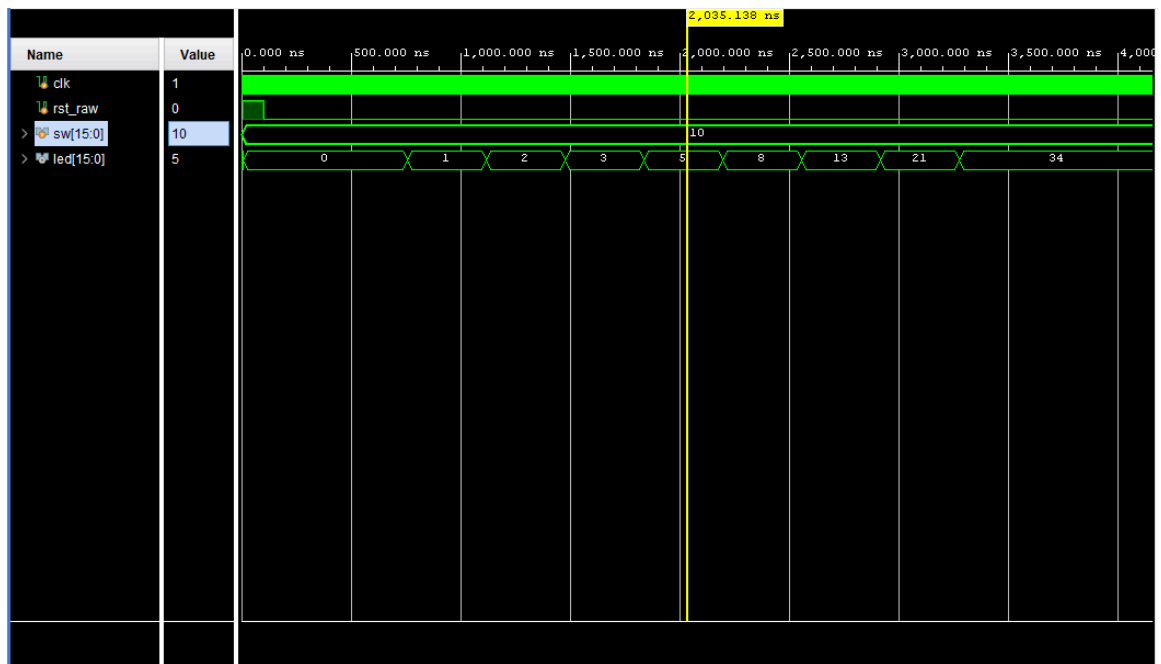
JAL verification in Task B

AUIPC Verification Logic: The AUIPC instruction was verified using a dedicated testbench (tb_AUIPC). In this test, an AUIPC instruction with an immediate value of 0x5321 was executed when the current Program Counter (PC) was 0x08. The Alu correctly calculated the target address x1 value as 0x5321008, confirming the hardware logic for PC-relative addressing is fully functional.



4. Task C: Program Execution with Updated Design

- **Execution Logic:** The Fibonacci assembly code was loaded into the fibonnaci.mem file. This program utilizes the newly implemented LUI and JAL instructions to manage loops and memory addressing.
- **Simulation Setup:** The simulation utilized the tb_Fibonacci testbench, which monitors the Data Memory array and the physical LED output signals.
- **Hardware Interaction:** The program calculated the sequence by reading a target 'n' from the hardware switches and iteratively storing results in the Data Memory.
- **Verification & Results:** The AUIPC instruction was successfully verified as it correctly calculated the relative offsets for memory storage. Correct execution was confirmed by inspecting the DataMemory array, which contained the sequence (0, 1, 1, 2, 3, 5, 8, 13, 21, 34) in positions mem[0] through mem[9].



5. Conclusion

The implementation of LUI, JAL, and AUIPC successfully extended the processor's capabilities, allowing for advanced procedure calls and efficient memory addressing. The lab demonstrated a clear understanding of processor design, instruction execution, and system-level integration. All modifications to the datapath and control unit were verified through comprehensive simulation and hardware execution.

Git Repository Link: https://github.com/emaanasad1217/CA_project.git